Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA)

Group Detail

Name: SARUKESH K

College: KPR College Of Arts Science And Research

Contact: 23bcs056@kprcas.ac.in

Name: SARAN S M

College: KPR College Of Arts Science And Research

Contact: 23bcs053@kprcas.ac.in

Name: BARATHNIVASH N

College: KPR College Of Arts Science And Research

Contact: 23bcs011@kprcas.ac.in

Name: PRASANTH K K

College: KPR College Of Arts Science And Research

Contact: 23bcs045@kprcas.ac.in

Project Title: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA)

Overview:

The principle of least-privilege access is a core concept in cybersecurity that ensures users, devices, and applications are granted only the minimum level of access necessary to perform their intended tasks. In conventional network security models, users often connect to internal resources through Virtual Private Networks (VPNs), which typically provide broad network-level access once authentication is completed. This model can expose organizations to significant security risks, including insider threats, credential misuse, and lateral movement by attackers who gain unauthorized entry into the network. Zscaler Private Access addresses these challenges by adopting a Zero Trust architecture that fundamentally changes how users access internal applications.

ZPA functions as a secure access broker that connects authenticated users directly to specific private applications without providing access to the underlying network infrastructure. The system uses identity-aware policies that evaluate user credentials, device security posture, and contextual information before granting access. Application-specific segmentation ensures that users can only reach the particular services or applications for which they have explicit authorization. This micro-segmentation approach prevents attackers from discovering or moving laterally across other resources even if a single account is compromised.

Another key advantage of ZPA is that it hides internal applications from direct internet exposure by eliminating inbound network connections. Applications remain invisible to unauthorized users because the platform establishes outbound-only connections to the Zscaler cloud. This architecture significantly reduces the attack surface and protects sensitive enterprise systems from common threats such as port scanning, exploitation attempts, and distributed denial-of-service attacks. Additionally, the cloud-native design of ZPA ensures high scalability, simplified management, and consistent policy enforcement across hybrid and multi-cloud environments.

Overall, Zscaler Private Access provides organizations with a modern, secure alternative to traditional remote access technologies. By enforcing least-privilege access through identity-driven and application-specific policies, ZPA strengthens security posture while delivering

seamless and reliable access to private applications for employees, partners, and remote users. This approach aligns with contemporary cybersecurity frameworks and supports organizations in building a resilient, Zero Trust-based digital infrastructure.

Abstract:

Zscaler Private Access (ZPA) is a cloud-delivered Zero Trust Network Access (ZTNA) solution designed to enforce strict least-privilege access to enterprise applications. Instead of granting users broad network connectivity as in traditional VPN architectures, ZPA provides secure, identity-based access only to specific applications that a user is authorized to use. By implementing identity-aware and application-specific access policies, the platform ensures that access decisions are based on user identity, device posture, location, and contextual risk factors. This approach eliminates the need to expose internal applications to the public internet, thereby reducing the attack surface and preventing unauthorized lateral movement within the network. Through continuous authentication and dynamic policy enforcement, ZPA establishes a secure, direct connection between authenticated users and private applications. As organizations adopt cloud computing, remote work environments, and distributed infrastructures, ZPA plays a crucial role in strengthening enterprise security by enabling scalable, flexible, and policy-driven access control aligned with modern Zero Trust principles.

Use Case Scenarios:

Scenario 1: Student Access to University Learning Management System

A university student needs to access the institution's Learning Management System (LMS) to submit assignments and view course materials. The student connects from their personal laptop using the ZPA Client Connector. After launching the client, the student authenticates through the university's identity provider using single sign-on (SSO) along with multi-factor authentication. ZPA evaluates the student's identity, role membership (Student_LMS_Group), device status, and login context. Based on the least-privilege access policy defined by the university IT department, the system grants access only to the LMS application. ZPA then establishes a secure micro-tunnel directly between the student and the LMS platform without exposing other internal academic systems. The student can download lecture notes and upload assignments but cannot access administrative databases or faculty systems.

Output:

- Student identity verified through SSO and multi-factor authentication
- Secure micro-tunnel created specifically for the LMS platform
- Other university internal systems remain hidden and inaccessible

Scenario 2: Marketing Team Access to Analytics Dashboard

A marketing analyst needs to review campaign performance metrics from the company's internal analytics dashboard. The analyst logs in from a corporate laptop while working remotely. Through the ZPA Client Connector, authentication occurs using corporate credentials integrated with the company's identity management platform. ZPA checks the user's group membership (Marketing_Analytics_Group), verifies the device posture, and evaluates contextual factors such as location and session risk level. Once validated, ZPA establishes a secure application-specific connection to the analytics dashboard. The analyst can analyze marketing campaign data and generate reports but cannot access other internal systems like financial accounting platforms or HR records.

Output:

- User authenticated and verified through identity-based access control
- Secure connection established only to the analytics dashboard
- Access to finance and HR systems remains restricted

Scenario 3: Research Team Access to Internal Data Repository

A research scientist needs to retrieve datasets from the organization's internal research data repository to conduct analysis. The scientist connects using a company-approved workstation and launches the ZPA Client Connector. After authentication through the corporate identity provider with MFA, ZPA verifies the scientist's role membership (Research_Data_Group) and confirms device compliance with security policies. Based on the configured least-privilege rules, ZPA allows the scientist to access only the research repository application. A secure micro-tunnel is created directly to the repository server while preventing discovery of other sensitive systems such as financial databases or operational servers.

Output:

• Identity and device posture validated before granting access
• Secure application-specific micro-tunnel established to the research repository
• Other enterprise systems remain invisible and protected

Scenario 4: Customer Support Agent Access to Ticketing System

A customer support agent working from a branch office needs to handle customer complaints using the organization's internal ticketing system. The agent logs in through ZPA using their corporate credentials and completes multi-factor authentication. ZPA verifies the agent's identity and confirms membership in the Support_Ticketing_Group. It also checks the device's compliance status and connection context. Once the policy engine approves the request, ZPA creates a secure micro-tunnel directly to the ticketing application. The agent can view and resolve support tickets but cannot access confidential systems such as payroll databases or internal development platforms.

Output:

• Agent authenticated through corporate identity provider and MFA
• Direct secure connection established to the ticketing application
• Sensitive internal systems remain inaccessible

Scenario 5: Project Manager Access to Project Management Portal

A project manager needs to monitor project progress using the organization's internal project management portal. The manager accesses the system from a company-issued laptop while traveling. After launching the ZPA Client Connector, authentication is completed through the organization's identity service with multi-factor authentication. ZPA evaluates the user's role membership (Project_Manager_Group), device posture, and contextual security parameters. Once verified, ZPA brokers a secure connection only to the project management portal. The manager can update project milestones and track team activities but cannot reach unrelated systems such as source code repositories or finance applications.

Output:

• Project manager authenticated and authorized through identity-based policies

• Secure micro-tunnel established exclusively for the project management portal

• Other internal enterprise resources remain hidden and inaccessible

STAGE-1

Top 10 OWASP:

No	Vulnerability Category	Vulnerability Name	CWE No
1	Injection	SQL Injection	CWE-89
2	Injection	Command Injection	CWE-77
3	Injection	Cross-Site Scripting (XSS)	CWE-79
4	Broken Authentication	Improper Authentication	CWE-287
5	Broken Authentication	Weak Password Requirements	CWE-521
6	Sensitive Data Exposure	Information Exposure Through Data Transmission	CWE-319
7	Security Misconfiguration	Missing Security Headers	CWE-693
8	Security Misconfiguration	Directory Listing Enabled	CWE-548

9	Broken Access Control	Improper Authorization	CWE-285
10	Broken Access Control	Insecure Direct Object Reference (IDOR)	CWE-639
11	Identification and Authentication Failures	Session Fixation	CWE-384
12	Software and Data Integrity Failures	Use of Components with Known Vulnerabilities	CWE-1104
13	Cryptographic Failures	Weak SSL/TLS Configuration	CWE-326
14	Logging and Monitoring Failures	Insufficient Logging and Monitoring	CWE-778
15	Server-Side Request Forgery	Server-Side Request Forgery (SSRF)	CWE-918

Reports:

1. SQL Injection

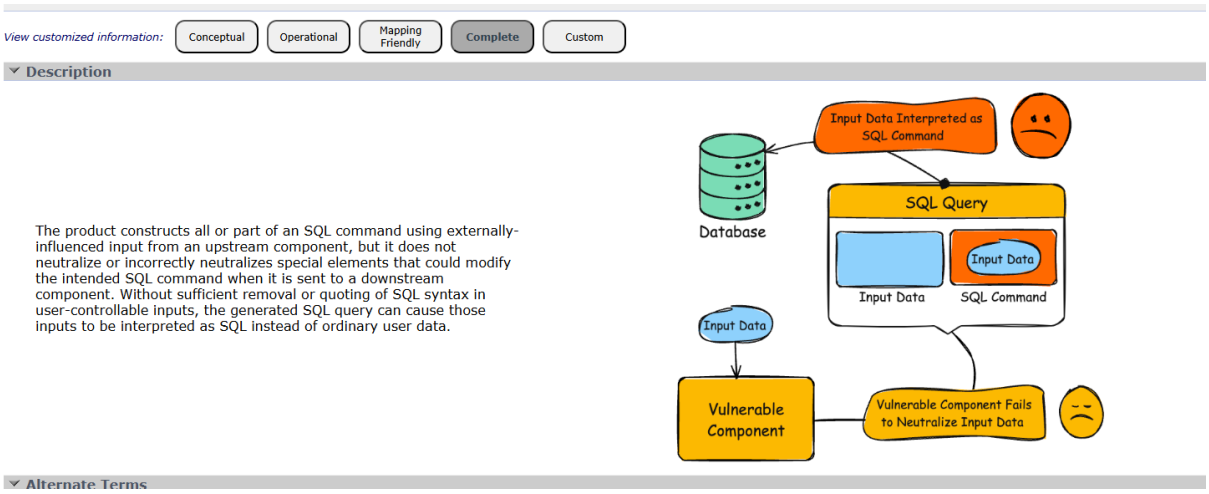
Vulnerability Name: SQL Injection
CWE/CVE: CWE-89
OWASP/SANS Category: Injection

Description:

SQL Injection occurs when an application includes untrusted user input in SQL queries without proper validation or sanitization. Attackers can manipulate database queries by injecting malicious SQL statements through input fields such as login forms, search fields, or URL parameters.

Business Impact:

Attackers may gain unauthorized access to sensitive data, modify or delete records, bypass authentication mechanisms, or completely compromise the database.



Recommendation:

Use parameterized queries or prepared statements, implement input validation, and use ORM frameworks. Avoid dynamic SQL queries and enforce least privilege database access.

2. Command Injection

Vulnerability Name: Command Injection

CWE/CVE: CWE-77

OWASP/SANS Category: Injection

Description:

Command Injection occurs when an application passes unsafe user input to system shell commands. Attackers can execute arbitrary commands on the server by injecting malicious input.

Business Impact:

Attackers may execute system commands, access sensitive files, modify system data, or gain full control of the server.

Improper Neutralization of Special Elements used in a Command ('Command Injection')

View on cwe.mitre.org

Recommendation:

Validate and sanitize user inputs, avoid executing system commands directly, and use safe APIs instead of shell commands. Implement least privilege for system processes.

3. Cross-Site Scripting (XSS)

Vulnerability Name: Cross-Site Scripting (XSS)

CWE/CVE: CWE-79

OWASP/SANS Category: Injection

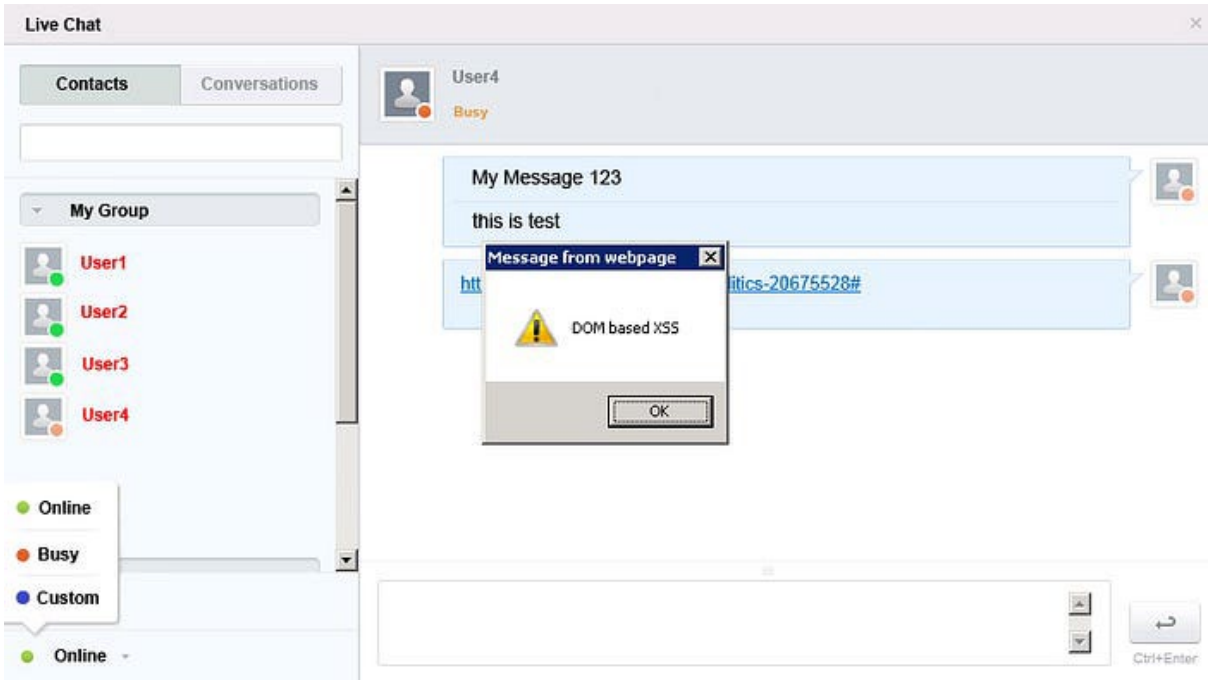
Description:

Cross-Site Scripting occurs when an application allows malicious scripts to be

injected into web pages viewed by other users. These scripts execute in the victim's browser.

Business Impact:

Attackers can steal session cookies, hijack user accounts, redirect users to malicious websites, or modify page content.



Recommendation:

Implement proper input validation and output encoding. Use Content Security Policy (CSP), secure cookies, and sanitize user-generated content.

4. Improper Authentication

Vulnerability Name: Improper Authentication

CWE/CVE: CWE-287

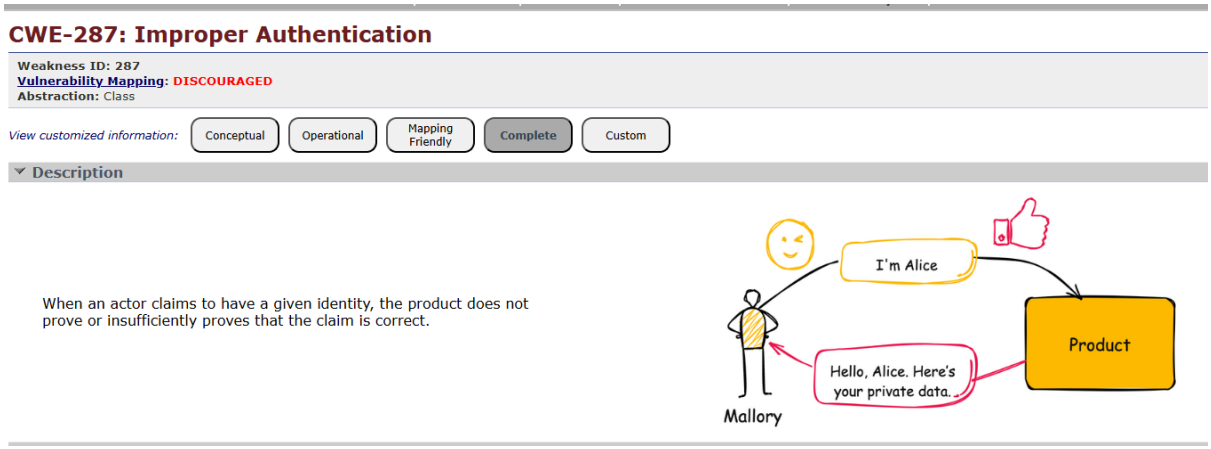
OWASP/SANS Category: Broken Authentication

Description:

Improper authentication occurs when an application fails to properly verify the identity of users attempting to access resources.

Business Impact:

Unauthorized users may gain access to protected resources, confidential data, or administrative functions.



Recommendation:

Implement strong authentication mechanisms, enforce secure login processes, and use multi-factor authentication where possible.

5. Weak Password Requirements

Vulnerability Name: Weak Password Requirements

CWE/CVE: CWE-521

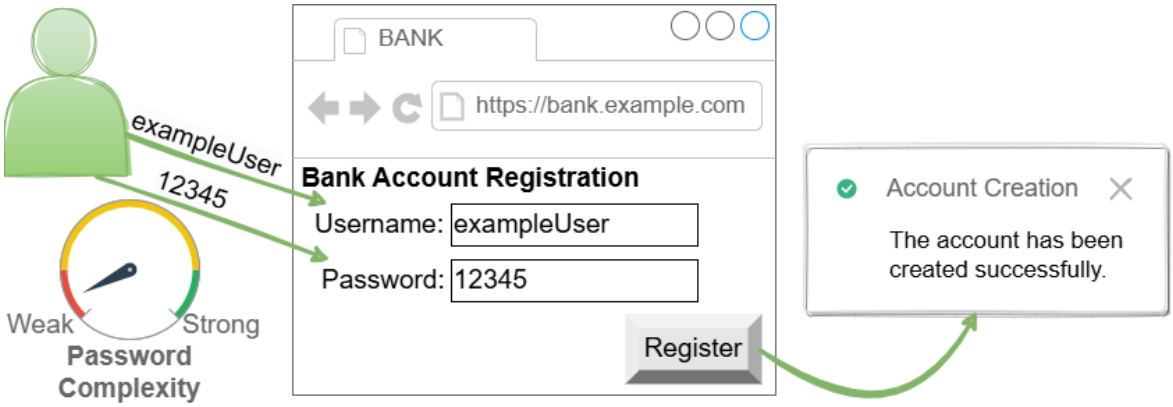
OWASP/SANS Category: Broken Authentication

Description:

Weak password policies allow users to create easily guessable passwords, making accounts vulnerable to brute force or dictionary attacks.

Business Impact:

Attackers may compromise user accounts and gain unauthorized access to systems and sensitive data.



Recommendation:

Enforce strong password policies including minimum length, complexity requirements, password expiration, and account lockout mechanisms.

6. Information Exposure Through Data Transmission

Vulnerability Name: Information Exposure Through Data Transmission

CWE/CVE: CWE-319

OWASP/SANS Category: Sensitive Data Exposure

Description:

Sensitive information is transmitted over the network without proper encryption, allowing attackers to intercept the data.

Business Impact:

Attackers may capture login credentials, personal information, or financial data.

Recommendation:

Use secure protocols such as HTTPS with strong TLS encryption to protect data during transmission.

7. Missing Security Headers

Vulnerability Name: Missing Security Headers

CWE/CVE: CWE-693

OWASP/SANS Category: Security Misconfiguration

Description:

Security headers help protect web applications from common attacks such as clickjacking and cross-site scripting. Missing headers reduce security protection.

Business Impact:

Attackers may exploit the application using browser-based attacks.

missing-security-headers-vuln-example

there is 5 vulnerabilities

1. Missing X-Frame-Options header in `index.vuln.js` line `1`
2. Missing X-Content-Type-Options header in `index.vuln.js` line `1`
3. Missing Content-Security-Policy header in `index.vuln.js` line `1`
4. Missing X-XSS-Protection header in `index.vuln.js` line `1`
5. Missing Strict-Transport-Security header in `index.vuln.js` line `1`

```
import express from "express";
import helmet from "helmet";
import bodyParser from "body-parser";

let app = express()
```

Recommendation:

Configure security headers such as Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security.

8. Directory Listing Enabled

Vulnerability Name: Directory Listing Enabled

CWE/CVE: CWE-548

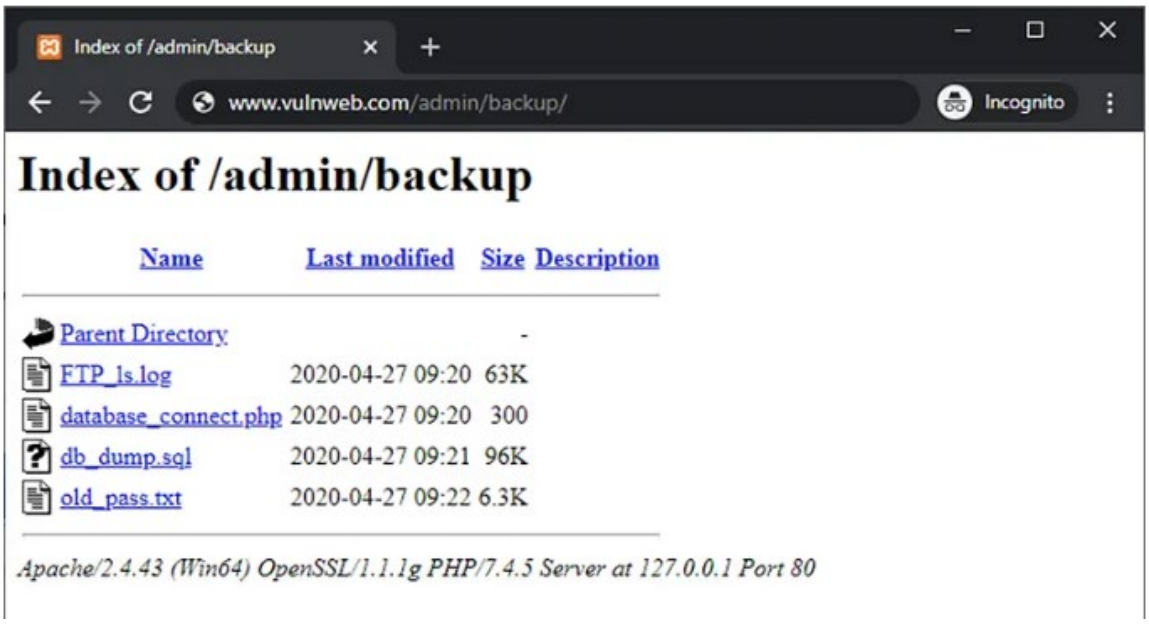
OWASP/SANS Category: Security Misconfiguration

Description:

Directory listing occurs when a web server allows users to view the contents of directories that should be restricted.

Business Impact:

Attackers may access sensitive files, configuration files, or source code.



Recommendation:

Disable directory listing on the web server and restrict access to sensitive directories.

9. Improper Authorization

Vulnerability Name: Improper Authorization

CWE/CVE: CWE-285

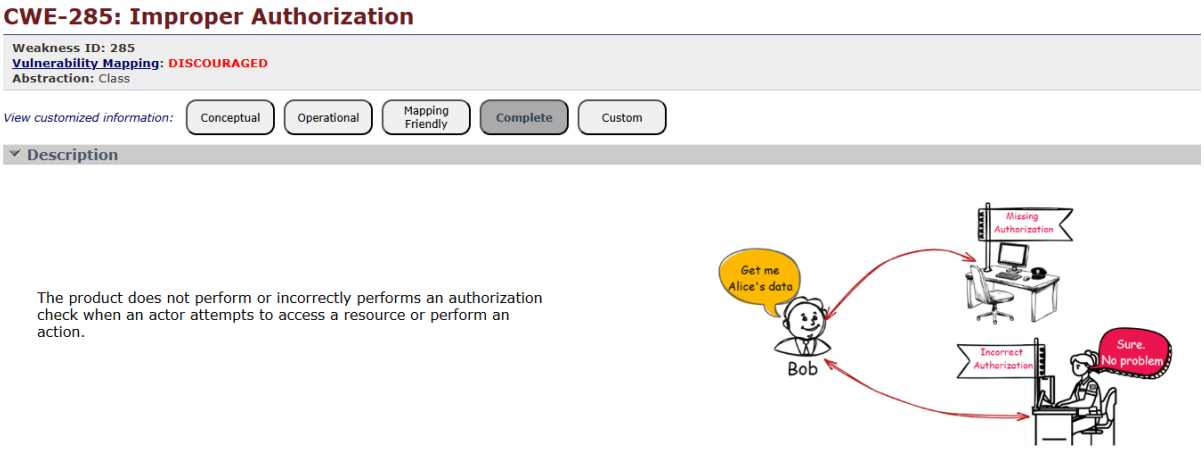
OWASP/SANS Category: Broken Access Control

Description:

Improper authorization occurs when an application does not properly verify whether a user has permission to perform a specific action.

Business Impact:

Attackers may gain access to restricted functionality or confidential data.



Recommendation:

Implement strong access control mechanisms and enforce role-based access control (RBAC).

10. Insecure Direct Object Reference (IDOR)

Vulnerability Name: Insecure Direct Object Reference (IDOR)

CWE/CVE: CWE-639

OWASP/SANS Category: Broken Access Control

Description:

IDOR occurs when an application exposes internal object references such as file names or database IDs without proper authorization checks.

Business Impact:

Attackers may access or modify other users' data by manipulating identifiers.

HTTP Copy

```
# The attacker is logged in as user 1234
https://example.org/user/id/1234

# The attacker changes the id in the URL and gains access to a different user
https://example.org/user/id/1235
```

For example, in the Express code below, the value given in the URL is available as `req.params.id`, and we use that value to retrieve the corresponding record in the database. We also check that the request is from an authenticated user, by (calling the `isAuthenticated` function). But critically, we don't check that the ID of the authenticated user matches the ID in the URL, and this enables one authenticated user (the attacker) to get a page for a different authenticated user (the victim).

JS Copy

```
app.get("/user/id/:id", (req, res) => {
  const user = db.users.find(req.params.id);
  if (req.isAuthenticated()) {
    // Authentication is not enough!
    res.render("user", { user });
  }
});
```

Recommendation:

Implement proper authorization checks and use indirect references instead of exposing direct object identifiers.

11. Session Fixation

Vulnerability Name: Session Fixation

CWE/CVE: CWE-384

OWASP/SANS Category: Identification and Authentication Failures

Description:

Session Fixation occurs when an attacker sets or predicts a user's session ID before authentication.

Business Impact:

Attackers may hijack user sessions and gain unauthorized access to accounts.

Code · 161 Bytes

```
SANDBOX-XSRF-TOKEN=AAG02cId-yyza3k8uhQR7JKuB-4YOmhizkjM;
romit.sandbox.session=s%3AEHm0kA9uwWYHayOwdRQXbuZWEIRIliQZ.ndejz36ofa52c9ENnApLuaLkMnTYCot3IiY1qdTvz0w;
```

Code · 889 Bytes

```
2. Now simulate the victim by opening a second browser and setting those two cookies.
3. As the victim, login in the second browser.
4. As the attacker, go to https://wallet.sandbox.romit.io (using the first browser / same cookies as in step 1). You are now logged in to the victims account.

Possible exploitation scenarios
---------------

This can be exploited if there is another bug like HTTP Response Splitting on your website.

But a far easier way is to exploit this on shared computers. For example in a library, as an attacker open https://wallet.sandbox.romit.io (but do not login!) and keep note of the cookies as above in step 1. Then simply go away and now when a victim will use the same computer and try to login, the attacker will have access to the victims account.

Mitigation
--------------

If you assign a new session when someone logs in, this flaw should be fixed.
```

Recommendation:

Regenerate session IDs after successful login and implement secure session management practices.

12. Use of Components with Known Vulnerabilities

Vulnerability Name: Use of Components with Known Vulnerabilities

CWE/CVE: CWE-1104

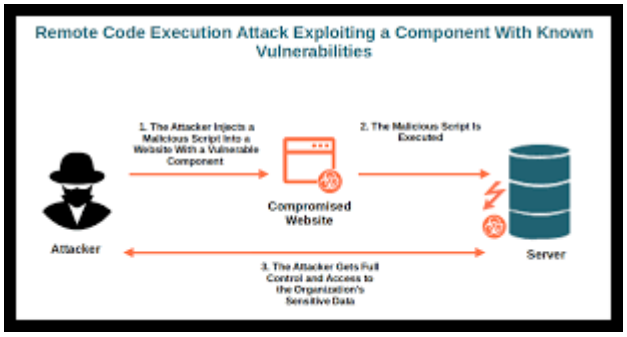
OWASP/SANS Category: Software and Data Integrity Failures

Description:

Applications often rely on third-party libraries and frameworks. If these components contain known vulnerabilities, attackers may exploit them.

Business Impact:

Systems may be compromised through outdated or vulnerable dependencies.



Recommendation:

Regularly update and patch third-party libraries and use vulnerability scanning tools.

13. Weak SSL/TLS Configuration

Vulnerability Name: Weak SSL/TLS Configuration

CWE/CVE: CWE-326

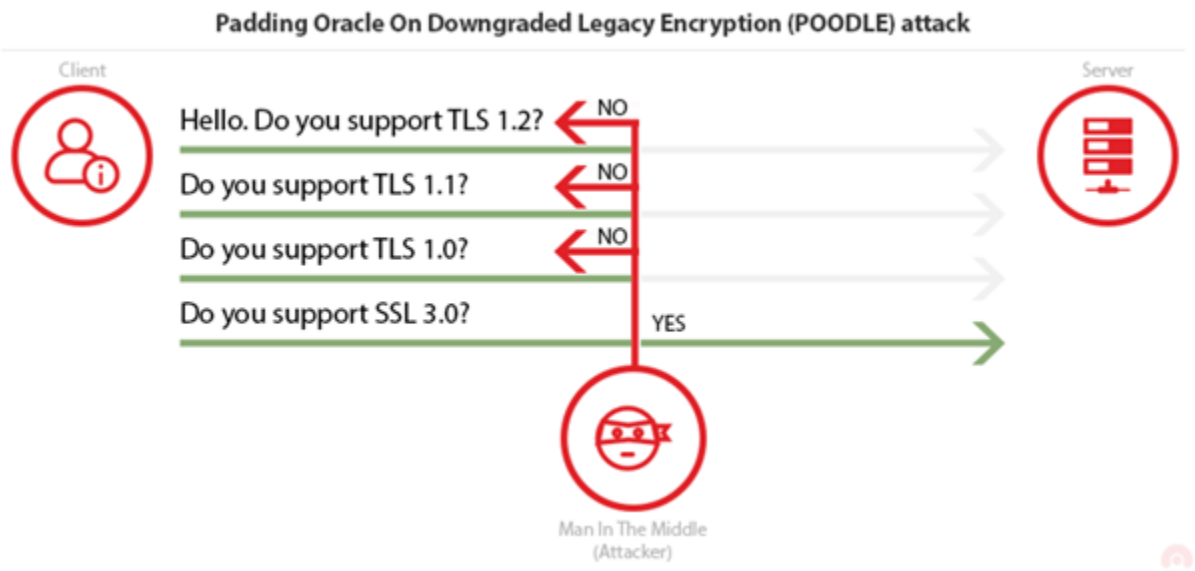
OWASP/SANS Category: Cryptographic Failures

Description:

Weak SSL/TLS configurations may allow attackers to decrypt or manipulate encrypted communication.

Business Impact:

Sensitive information may be exposed during transmission.



Recommendation:

Use strong encryption protocols (TLS 1.2 or higher) and disable outdated protocols such as SSLv2, SSLv3, and TLS 1.0.

14. Insufficient Logging and Monitoring

Vulnerability Name: Insufficient Logging and Monitoring

CWE/CVE: CWE-778

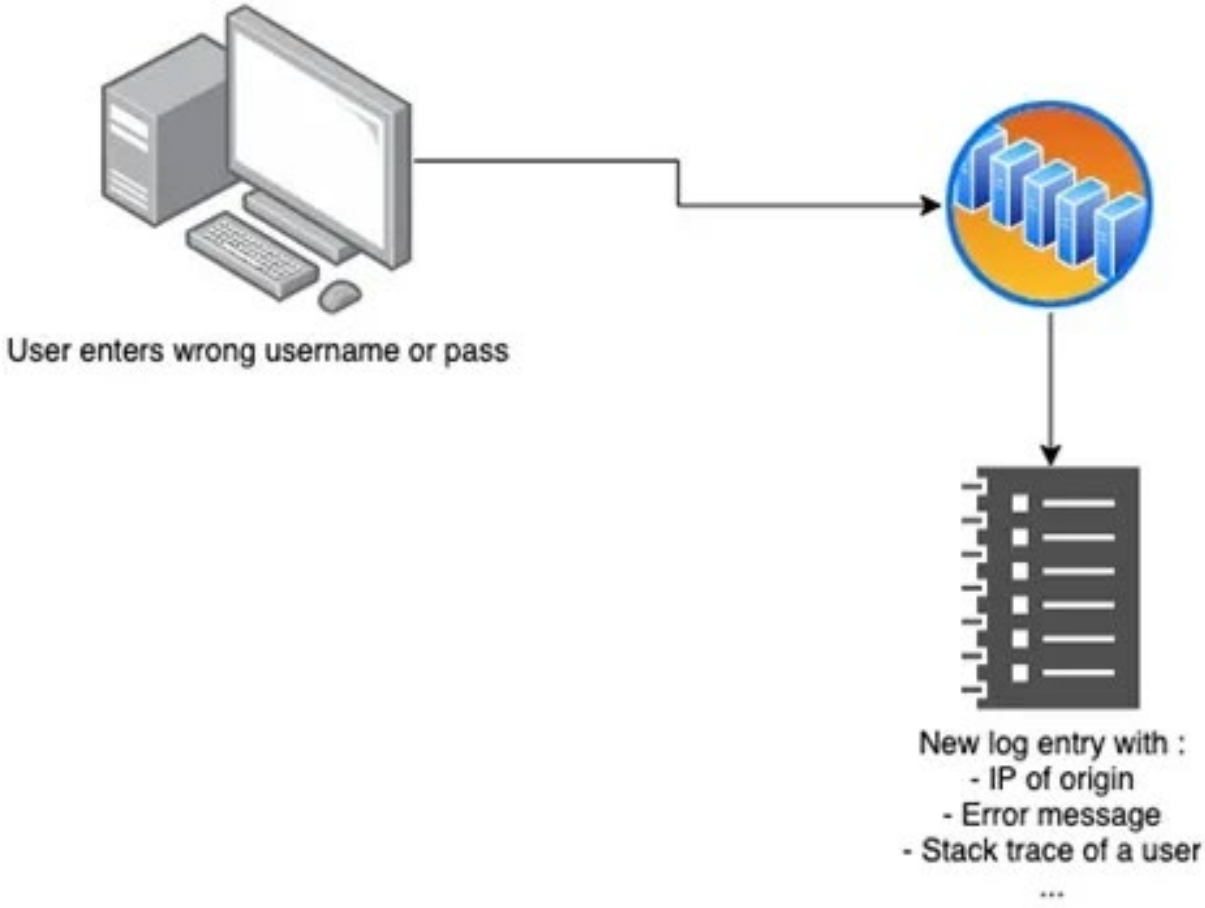
OWASP/SANS Category: Logging and Monitoring Failures

Description:

Applications without proper logging and monitoring cannot detect suspicious activities or security breaches.

Business Impact:

Security incidents may go unnoticed for long periods, increasing damage.



Recommendation:

Implement centralized logging, monitor security events, and configure alert mechanisms.

15. Server-Side Request Forgery (SSRF)

Vulnerability Name: Server-Side Request Forgery (SSRF)

CWE/CVE: CWE-918

OWASP/SANS Category: Server-Side Request Forgery

Description:

SSRF occurs when an attacker manipulates a server to send requests to internal or external systems.

Business Impact:

Attackers may access internal services, sensitive data, or cloud metadata.

CVE-2026-31829 Detail

Description

Flowise is a drag & drop user interface to build a customized large language model flow. Prior to 3.0.13, Flowise exposes an HTTP Node in AgentFlow and Chatflow that performs server-side HTTP requests using user-controlled URLs. By default, there are no restrictions on target hosts, including private/internal IP ranges (RFC 1918), localhost, or cloud metadata endpoints. This enables Server-Side Request Forgery (SSRF), allowing any user interacting with a publicly exposed chatflow to force the Flowise server to make requests to internal network resources that are inaccessible from the public internet. This vulnerability is fixed in 3.0.13.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD	**Base Score:** 8.8 HIGH	**Vector:** CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CNA: GitHub, Inc.	**Base Score:** 7.1 HIGH	**Vector:** CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:L

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on

Recommendation:

Validate and restrict URL inputs, implement network segmentation, and block access to internal resources.

STAGE-2

Nessus

Overview:

Nessus is a popular vulnerability assessment tool developed by Tenable that is used to identify security weaknesses in computer systems, networks, servers, and web applications. It helps organizations detect vulnerabilities such as outdated software, missing security patches, weak configurations, open ports, and potential malware that attackers could exploit. Nessus works by scanning the target systems and comparing the results with its continuously updated database of known vulnerabilities and security issues.

The tool uses a large collection of plugins to perform different types of security checks and vulnerability tests. These plugins allow Nessus to detect thousands of known vulnerabilities across various operating systems, databases, applications, and network devices. After completing a scan, Nessus generates detailed reports that categorize vulnerabilities based on their severity levels such as critical, high, medium, and low. The reports also include a description of each vulnerability, its potential impact, and recommended solutions or remediation steps to fix the problem.

Nessus supports multiple scanning techniques including network scanning, configuration auditing, malware detection, and web application vulnerability scanning. It also allows users to schedule scans, customize scanning policies, and monitor system security regularly. Because of its high accuracy, user-friendly interface, and regular updates, Nessus is widely used by cybersecurity professionals, penetration testers, and organizations to strengthen their security posture and protect systems from potential cyber threats

use cases of Nessus:

- Vulnerability Assessment
  Nessus is widely used to identify security vulnerabilities in networks, servers, and systems. It scans devices to detect weaknesses such as outdated software, unpatched systems, and insecure configurations that attackers could exploit.
- Network Security Monitoring
  Organizations use Nessus to monitor their network security by scanning routers,

switches, firewalls, and other network devices. This helps security teams detect open ports, weak protocols, and other network-related vulnerabilities.

- Compliance Auditing
  Nessus helps organizations meet security compliance requirements such as PCI-DSS, HIPAA, and ISO standards. It checks systems against security policies and compliance guidelines to ensure that organizations follow required security practices.
- Patch Management Support
  Nessus identifies missing patches and outdated software versions on systems. This allows administrators to update and patch their systems quickly to reduce security risks.
- Web Application Security Testing
  Security professionals use Nessus to scan web applications for vulnerabilities like SQL injection, cross-site scripting (XSS), and other common web security issues.
- Configuration Auditing
  Nessus checks system configurations to identify weak settings such as default passwords, unnecessary services, or improper security configurations that could lead to security breaches.
- Penetration Testing Support
  Ethical hackers and penetration testers use Nessus as a reconnaissance tool to gather vulnerability information before performing deeper security testing.
- Risk Management
  Nessus provides vulnerability severity ratings and detailed reports, helping organizations prioritize security issues and fix the most critical vulnerabilities first.

Features of Nessus

1. **Comprehensive Vulnerability Detection**
   Nessus can detect thousands of vulnerabilities in operating systems, network devices, databases, and web applications. It uses a large and frequently updated plugin database to identify security weaknesses.
2. **High-Speed Asset Discovery**
   The tool quickly scans networks to discover active hosts, open ports, and running services. This helps security teams understand what devices are connected to the network.

3. **Configuration Auditing**
   Nessus checks system configurations and security settings to ensure they follow recommended security standards. It can detect weak configurations such as default credentials or insecure protocols.
4. **Malware Detection**
   Nessus helps identify systems that may be infected with malware or communicating with suspicious or malicious hosts.
5. **Web Application Scanning**
   It scans web applications for common vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and other security flaws.
6. **Detailed Vulnerability Reports**
   After scanning, Nessus generates detailed reports that include vulnerability descriptions, severity levels (Critical, High, Medium, Low), and recommended solutions.
7. **Customizable Scan Policies**
   Users can create customized scanning policies depending on the network environment and security requirements.
8. **Regular Plugin Updates**
   Nessus receives regular updates from Tenable to detect newly discovered vulnerabilities.

How to Use Nessus

1. **Install Nessus**
   Download and install Nessus from the official website of Tenable. It is available for Windows, Linux, and macOS systems.
2. **Activate the Tool**
   After installation, activate Nessus using an activation code provided during registration.
3. **Access the Web Interface**
   Nessus runs through a web-based interface. Open a browser and access it using: https://localhost:8834
4. **Create a New Scan**
   Click **New Scan** and choose a scan template such as **Basic Network Scan**.

5. **Configure the Scan**
   Enter the **target IP address or domain name** of the system you want to scan and configure scan settings if required.
6. **Run the Scan**
   Start the scan. Nessus will analyze the target system and check for vulnerabilities.
7. **Review the Results**
   After the scan is completed, Nessus displays the vulnerabilities found along with their severity levels.
8. **Generate Reports**
   Export the scan results as reports (PDF, HTML, or CSV) for documentation and further security analysis.

Target website: https://public-firing-range.appspot.com/

Target ip address: 172.217.26.20

S.NO	Vulnerability Name	Severity	Plugin Id
1	Traceroute Information	General	10287
2	TLS NPN Supported	Misc.	87242
3	TCP/IP Timestamp	General	25220
4	Shor’s Harvest Now Decrypt	General	29387
5	Remote Service Using Post-Quantum	General	277650
6	QUIC Service Detection	Service Detection	206982
7	OS Identification	General	11936
8	OS Fingerprints Detected	General	209654
9	OpenSSL Detection	Service Detection	50845
10	Nessus SYN Scanner	Port Scanner	11219

Reports:

1. Vulnerability Name:- Traceroute Information

Severity :- General
Plugin :- Traceroute Information
Plugin ID :- 10287
Port :- Multiple network hops detected

Description:-

The Traceroute Information vulnerability indicates that the scanning tool was able to successfully perform a traceroute operation to the target system. Traceroute is a network diagnostic technique used to identify the path that packets travel from the source system to the destination server. During this process, the scanning system receives responses from intermediate network devices such as routers, gateways, and firewalls.

While traceroute is commonly used for troubleshooting and network analysis, it can unintentionally reveal valuable information about the network infrastructure. Attackers can use the traceroute results to identify network topology, intermediate routing devices, and potential security controls such as firewalls and load balancers.

This information may help attackers map the network structure and identify weak points within the infrastructure that could be targeted in future attacks.

Solution:-

• Configure routers and firewalls to limit ICMP responses used by traceroute
• Disable unnecessary diagnostic services on external network interfaces
• Implement network filtering rules to block unauthorized traceroute requests
• Monitor network traffic for suspicious probing activity

Business Impact:-

Exposure of network routing information may assist attackers in understanding the internal network layout. This information can be used to plan targeted attacks against specific infrastructure components such as routers, gateways, or firewalls. Over time, this can increase the risk of network reconnaissance, targeted exploitation, and potential service disruption.

2. Vulnerability Name:- TLS NPN Supported

Severity :- Misc

Plugin :- TLS Next Protocol Negotiation Supported

Plugin ID :- 87242

Port :- 443

Description:-

TLS Next Protocol Negotiation (NPN) is a TLS extension used to negotiate application-layer protocols between a client and a server during the TLS handshake. Although NPN was originally introduced to support protocol negotiation for technologies such as HTTP/2, it has been largely replaced by the more modern Application Layer Protocol Negotiation (ALPN) mechanism.

If a server still supports TLS NPN, it may indicate outdated or legacy TLS configurations. Older TLS negotiation mechanisms can potentially introduce compatibility issues or security weaknesses if not properly maintained. Attackers may use this information to determine whether the system relies on outdated encryption protocols.

Solution:-

• Disable TLS NPN support if it is not required
• Upgrade TLS configurations to use ALPN instead
• Keep SSL/TLS libraries updated to the latest versions
• Conduct regular TLS configuration assessments

Business Impact:-

Legacy TLS features may weaken the overall security posture of the organization. Attackers could potentially exploit outdated cryptographic implementations to perform downgrade attacks or intercept encrypted communications. Maintaining modern TLS configurations is essential for protecting sensitive data in transit.

3. Vulnerability Name:- TCP/IP Timestamp

Severity :- General

Plugin :- TCP Timestamp Detection

Plugin ID :- 25220

Port :- Multiple ports

Description:-

TCP timestamps are optional fields included in TCP packet headers that provide information about packet timing and round-trip latency. These timestamps are typically used to improve network performance and detect duplicate packets.

However, the presence of TCP timestamps can also reveal information about the target system's uptime and clock behavior. Attackers may analyze these timestamps to determine how long a system has been running without rebooting, which can provide insights into patch cycles and maintenance schedules.

Solution:-

• Disable TCP timestamps on publicly accessible servers
• Apply network filtering rules to limit exposure of TCP header information
• Use firewall policies to restrict unnecessary network responses

Business Impact:-

Although this vulnerability does not directly compromise system security, it may provide attackers with information that can be used during reconnaissance phases. Knowledge about system uptime and behavior can assist attackers in planning more effective exploitation strategies.

4. Vulnerability Name:- Shor's Harvest Now Decrypt Later

Severity :- General
Plugin :- Quantum Cryptography Risk Detection
Plugin ID :- 29387
Port :- 443

Description:-

This vulnerability highlights the potential risk posed by quantum computing to traditional cryptographic algorithms. Current encryption algorithms such as RSA and ECC rely on mathematical problems that are difficult for classical computers to solve. However, future quantum computers could potentially break these encryption schemes using algorithms such as Shor's algorithm.

The "Harvest Now, Decrypt Later" attack strategy involves adversaries capturing encrypted data today with the intention of decrypting it in the future when quantum computing technology becomes sufficiently advanced.

Solution:-

• Begin adopting post-quantum cryptographic algorithms
• Monitor developments in quantum-safe encryption standards
• Implement hybrid cryptographic approaches where possible

Business Impact:-

Sensitive information transmitted today could be stored by attackers and decrypted in the future once quantum computing capabilities become available. Organizations handling long-term confidential data should begin planning migration to quantum-resistant encryption methods.

5. Vulnerability Name:- Remote Service Using Post-Quantum

Severity :- General
Plugin :- Post-Quantum Cryptography Detection
Plugin ID :- 277650
Port :- 443

Description:-

This plugin detects whether remote services support post-quantum cryptographic algorithms. Post-quantum cryptography refers to encryption methods designed to remain secure even against attacks from quantum computers.

The detection indicates that the service may be using experimental or emerging cryptographic protocols that aim to resist future quantum attacks.

Solution:-

• Evaluate the stability and compatibility of post-quantum cryptographic implementations
• Monitor security standards for approved post-quantum algorithms
• Maintain backward compatibility with existing secure protocols

Business Impact:-

Early adoption of post-quantum encryption technologies can improve long-term data security. However, experimental implementations must be carefully evaluated to ensure they do not introduce new vulnerabilities.

6. Vulnerability Name:- QUIC Service Detection

Severity :- Service Detection

Plugin :- QUIC Protocol Detection

Plugin ID :- 206982

Port :- 443 / UDP

Description:-

QUIC (Quick UDP Internet Connections) is a transport layer protocol designed to improve web performance and reduce connection latency. It operates over UDP instead of TCP and is commonly used with HTTP/3.

The detection of QUIC indicates that the server supports this protocol for secure and efficient communication.

Solution:-

• Ensure that QUIC implementations are updated regularly
• Monitor for vulnerabilities in HTTP/3 and QUIC protocol implementations
• Apply appropriate firewall rules for UDP traffic

Business Impact:-

While QUIC improves performance and security, misconfigured implementations could introduce vulnerabilities or expose additional attack surfaces.

7. Vulnerability Name:- OS Identification

Severity :- General

Plugin :- OS Identification

Plugin ID :- 11936

Port :- Multiple ports

Description:-

The scanner was able to identify the operating system running on the target host. OS detection typically relies on analyzing network responses, TCP/IP stack behavior, and packet characteristics.

Solution:-

• Configure firewalls to limit OS fingerprinting attempts
• Implement intrusion detection systems

Business Impact:-

Attackers may use OS identification information to target known vulnerabilities associated with specific operating systems.

8. Vulnerability Name:- OS Fingerprints Detected

Severity :- General
Plugin :- OS Fingerprinting
Plugin ID :- 209654
Port :- Multiple ports

Description:-

OS fingerprinting allows attackers to determine the operating system type and version running on the target host by analyzing network packet responses.

Solution:-

• Implement network filtering policies
• Limit system information disclosure

Business Impact:-

Knowledge of operating system details allows attackers to focus on vulnerabilities specific to that OS.

9. Vulnerability Name:- OpenSSL Detection

Severity :- Service Detection
Plugin :- OpenSSL Detection
Plugin ID :- 50845
Port :- 443

Description:-

The scanner detected the presence of the OpenSSL cryptographic library on the remote server. OpenSSL is widely used to implement SSL/TLS encryption for secure communications.

Solution:-

• Ensure OpenSSL is updated to the latest secure version
• Apply patches for known vulnerabilities

Business Impact:-

Outdated OpenSSL versions may contain vulnerabilities such as buffer overflows or cryptographic weaknesses that attackers can exploit.

10. Vulnerability Name:- Nessus SYN Scanner

Severity :- Port Scanner

Plugin :- Nessus SYN Scanner

Plugin ID :- 11219

Port :- Multiple ports

Description:-

This plugin indicates that the scan was performed using a SYN scanning technique. SYN scanning sends TCP SYN packets to identify open ports on the target system.

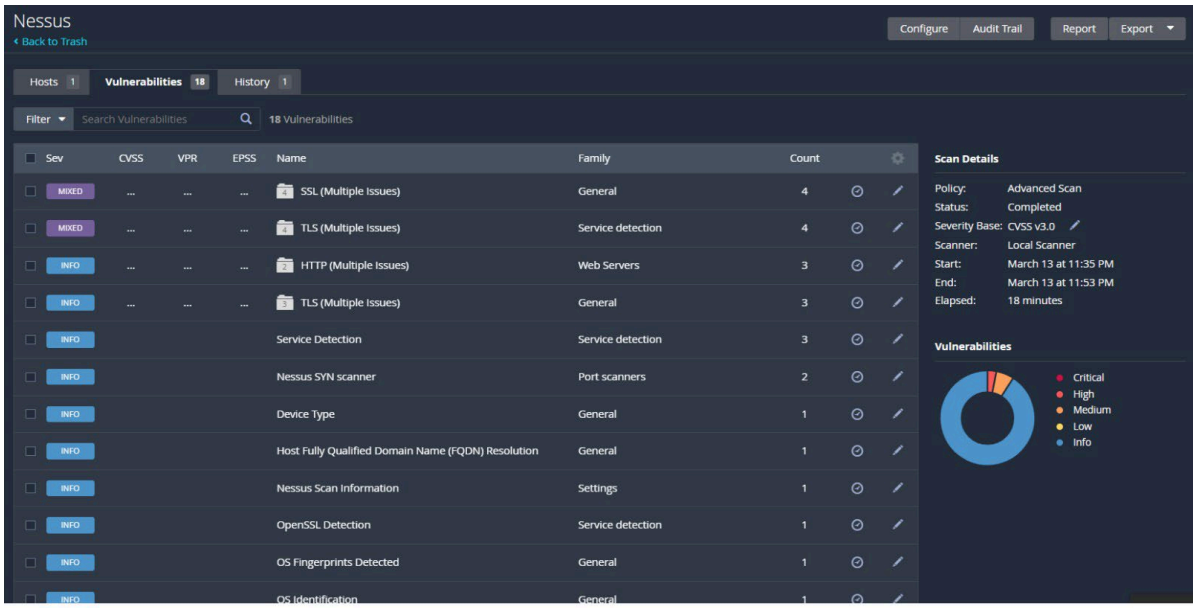
Solution:-

• Configure intrusion detection systems to detect scanning activities

• Implement firewall rules to limit unauthorized scanning

Business Impact:-

Port scanning may reveal open services running on the system. Attackers can use this information to identify potential entry points for exploitation.



STAGE-3

REPORT

Title: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

1. Principle of Least-Privilege Access in Modern Cybersecurity

The principle of least privilege (PoLP) is a fundamental security concept that ensures users, applications, and systems are granted only the minimum level of access necessary to perform their required tasks. This approach significantly reduces the attack surface within an organization's IT environment. Instead of giving broad network access, least-privilege models restrict permissions so that users can interact only with specific resources relevant to their roles.

2. Identity-Aware Access Control

Identity-aware access control focuses on verifying the identity of users before granting them access to internal applications or services. Instead of relying solely on traditional network-based security mechanisms such as IP addresses or VPN connections, identity-aware systems evaluate who the user is, what device they are using, and whether the access request meets defined security policies.

3. Application-Specific Access Policies

Application-specific access policies are designed to control access at the application level rather than granting broad network connectivity. In traditional security architectures, users who connect to a corporate network through a VPN often gain visibility into large portions of the internal infrastructure. This approach increases the risk of unauthorized access and lateral movement if a system becomes compromised.

Application-specific policies eliminate this risk by granting users access only to the exact applications they are permitted to use. For example, a finance employee may be allowed to access an enterprise resource planning (ERP) system but not development servers or internal databases. Each access request is evaluated against predefined policies that determine whether the user is authorized to connect to the requested application.

4. Secure Application Access Using Zscaler Private Access (ZPA)

Secure application access can be effectively implemented using Zscaler Private Access, which is a cloud-based platform designed to provide secure connectivity to internal applications without exposing the corporate network to the public internet. ZPA follows the Zero Trust security model, where access is granted based on verified identity and security posture rather than network location.

Conclusion:

Stage 1:

Stage 1 of the project focused on understanding the fundamental concepts of web application security and identifying common vulnerabilities present in publicly accessible web applications. During this phase, reconnaissance and vulnerability scanning techniques were applied to analyze the security posture of the target web environment. Several informational and service detection vulnerabilities such as Traceroute Information, TCP/IP Timestamp exposure, OS fingerprinting, and OpenSSL detection were identified through automated scanning tools.

This stage helped in understanding how attackers gather preliminary information about a target system before launching more complex attacks. It also provided insight into how minor information disclosure vulnerabilities can assist attackers in mapping the system architecture and identifying potential entry points. The analysis performed in this stage emphasized the importance of proper network configuration, secure service management, and minimizing unnecessary information exposure. Overall, Stage 1 laid the foundation for deeper vulnerability analysis and exploitation performed in the later stages of the project.

Stage 2:

Stage 2 concentrated on identifying and analyzing application-level vulnerabilities within the target web application environment. By examining the behavior of the application and performing controlled testing, several security weaknesses were observed that could potentially compromise system confidentiality, integrity, and availability.

This stage emphasized the importance of secure coding practices and proper input validation mechanisms. The analysis demonstrated how attackers can exploit poorly implemented authentication mechanisms, insecure configurations, and outdated protocols to gain unauthorized access or gather sensitive information. The testing process also highlighted how

automated vulnerability scanners such as network scanners and web application scanners assist security professionals in identifying weaknesses efficiently.

Stage 3:

Stage 3 focused on documenting and analyzing the vulnerabilities identified during the scanning and testing phases. Each vulnerability was studied in detail, including its description, technical cause, severity level, and potential business impact. Appropriate remediation strategies were also proposed to help mitigate the identified risks.

This stage highlighted the importance of vulnerability management and proper security documentation in cybersecurity operations. By systematically documenting vulnerabilities, organizations can prioritize remediation efforts based on risk severity and business impact. The process also demonstrated how structured reporting helps stakeholders clearly understand security risks and take appropriate corrective actions.

Future Scope:

Stage 1:

The work performed in Stage 1 can be extended further by implementing advanced reconnaissance and network analysis techniques. Future work may involve integrating automated security monitoring tools to continuously detect network exposure and service misconfigurations. Organizations can also deploy advanced intrusion detection and prevention systems to monitor suspicious scanning activities and unauthorized probing attempts.

Additionally, integrating threat intelligence feeds and real-time vulnerability databases could enhance the detection of newly emerging threats. Future research may also explore the implementation of zero trust networking models and identity-aware access controls to reduce the attack surface exposed to external entities.

Stage 2:

Future improvements for this stage may include performing advanced penetration testing techniques such as manual exploitation, fuzz testing, and API security assessments. Security engineers may also integrate secure development lifecycle practices into the application development process to detect vulnerabilities during early development stages.

Another potential extension is the implementation of automated vulnerability management systems that continuously monitor applications for security flaws. Machine learning–based

threat detection systems could also be explored to identify abnormal application behaviors that may indicate exploitation attempts.

Stage 3:

Future work in this stage can focus on implementing continuous vulnerability management frameworks that automatically detect, prioritize, and remediate vulnerabilities. Organizations may also integrate automated patch management systems to reduce the time required to fix security flaws.

Additionally, advanced risk assessment models could be developed to quantify vulnerability impact more accurately. Security teams may also integrate security dashboards and real-time reporting systems to provide better visibility into the organization's overall cybersecurity posture.

Topics Explored

- Web Application Security Fundamentals
- Network Reconnaissance Techniques
- Vulnerability Assessment Methodologies
- Information Disclosure Vulnerabilities
- Secure Communication Protocols
- Cryptographic Security Concepts
- Operating System Fingerprinting Techniques
- Service Detection and Network Scanning
- Vulnerability Management and Risk Analysis
- Cybersecurity Best Practices for Web Applications

Tools Explored

- Nessus Vulnerability Scanner
- Web Browser Developer Tools
- Network Scanning Utilities
- Traceroute and Network Diagnostic Tools
- Vulnerability Assessment Platforms

- Security Testing Environments
- Web Application Testing Frameworks